[파일 3] 참고문헌 및 참고자료 묶음

부문 3 활용 성과 촉진 아이디어 제안서 「효능 내비게이터」의 근거 자료 · 화면 캡처 2026-10-12 · 2026-10-22

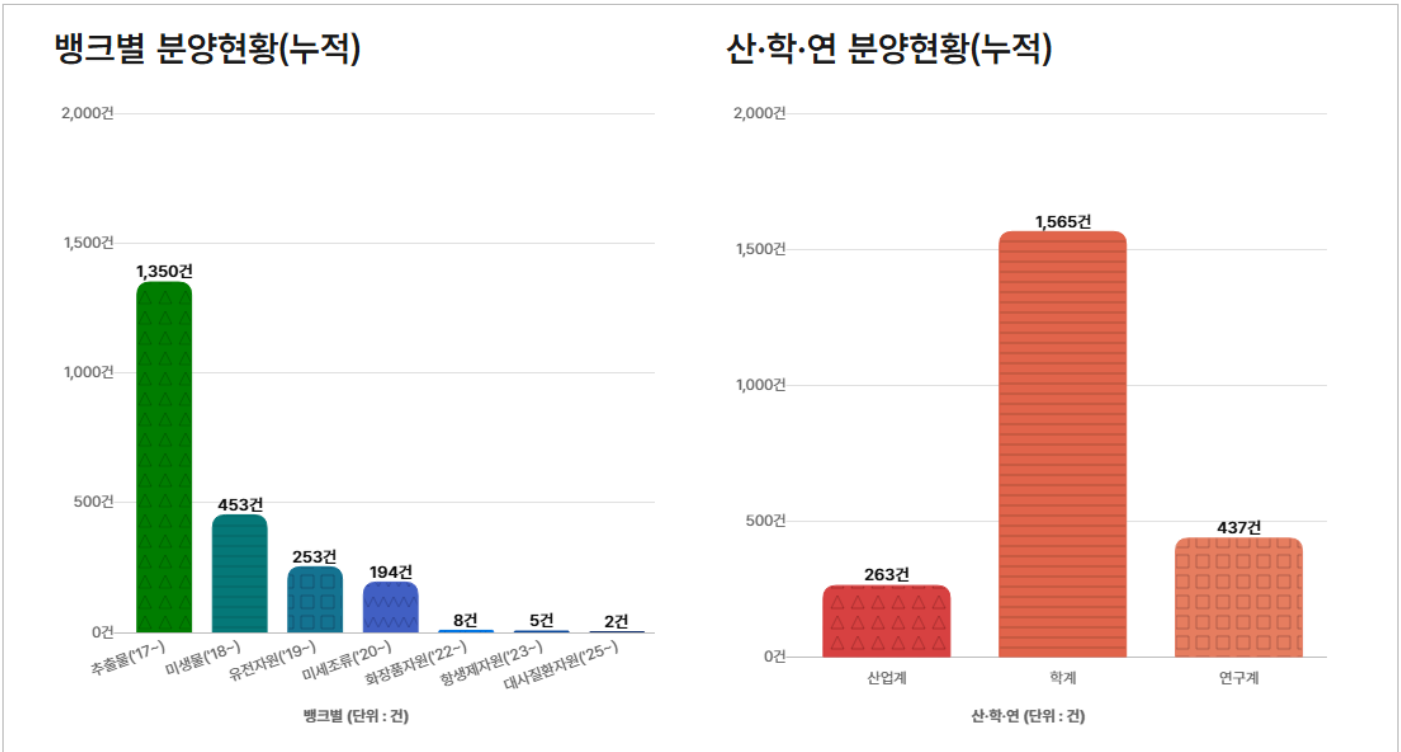
구분	쪽	자료	출처 · 기준일	본문 각주
근거 자료	2	**MBRIS 통계현황** 뱅크별·산학연 누적 분양, 보유자원	국립해양생물자원관 MBRIS (2025.12.31. · 2026.10.12.)	1~3, 23
	3	**MBRIS 분양 안내 · 성과현황** 분양승인 기준, 회당·연간 한도, 6개월 후 성과보고서 제출, 성과현황 게시판 0건	MBRIS (2026.10.21.~22. 확인)	9, 10, 22
	4	**MBRIS 공지 · 운영 전략** 분양 접수 기간·수령 일자, 누리집 리뉴얼, 수요자 맞춤형 분양 서비스	공지 2026.8.21. · 2026.1.8.	4, 9, 19
	5	**MBRIS Q&A · 효능 검색** Q&A 게시판 12건과 공개 게시글 발췌, 화장품자원 효능 검색 화면	MBRIS (2026.10.12. · 10.22.)	4, 7
	6	**Mbio-Bridge** 해양바이오 주요소재 소개(6건), 기업수요 매칭	국립해양생물자원관	6
	7	**학술 문헌** Saslis-Lagoudakis 외(2012), Schober 외(2025) 서지 정보와 초록	PNAS 109(39) · Nucleic Acids Res 53(D1)	11, 15, 16
	8	**해외 사례 · 인건비 근거** NCI 천연물 분획 라이브러리, 2026년 적용 SW기술자 평균임금	미국 국립암연구소 · 한국인공지능·소프트웨어산업협회 (2025.12.19.)	17, 21
	9	**언론 보도** 호남권생물자원관 AI 시스템, 국가 바이오데이터 구축, 해양바이오 상용화 국감 지적	데일리안 · 아시아경제 · 푸드투데이	18, 24, 25
	10	**공공데이터포털** 해양생물 채집 공간정보, 해양생물 화장품효능	국립해양생물자원관 (공공데이터포털)	분석 입력 자료
부록	11~12	**부록 A. 분석 방법과 결과 요약**	제안서 수치의 산출 근거	5, 8, 12~14, 20
	13	**부록 B. 재현 정보와 사용한 공개 자료 목록**	자료 9종 · 실행 환경 · 순서	–
	14~15	**부록 C. 결과 표**	시험별 A등급·AUC, 계통-효능 지도, 50주 모의, 미시험 현황	12~14
	16~24	**부록 D. 분석 코드 전문**	7개 파일, 줄 번호 표기	–

※ 본문 각주 번호를 따라 해당 쪽에서 원문을 확인할 수 있음
※ 학술 논문은 원문 사이트의 자동 접근 차단으로 서지 정보와 초록을 정리하여 수록함
※ 개인 연락처(이메일·전화)는 가림 처리함

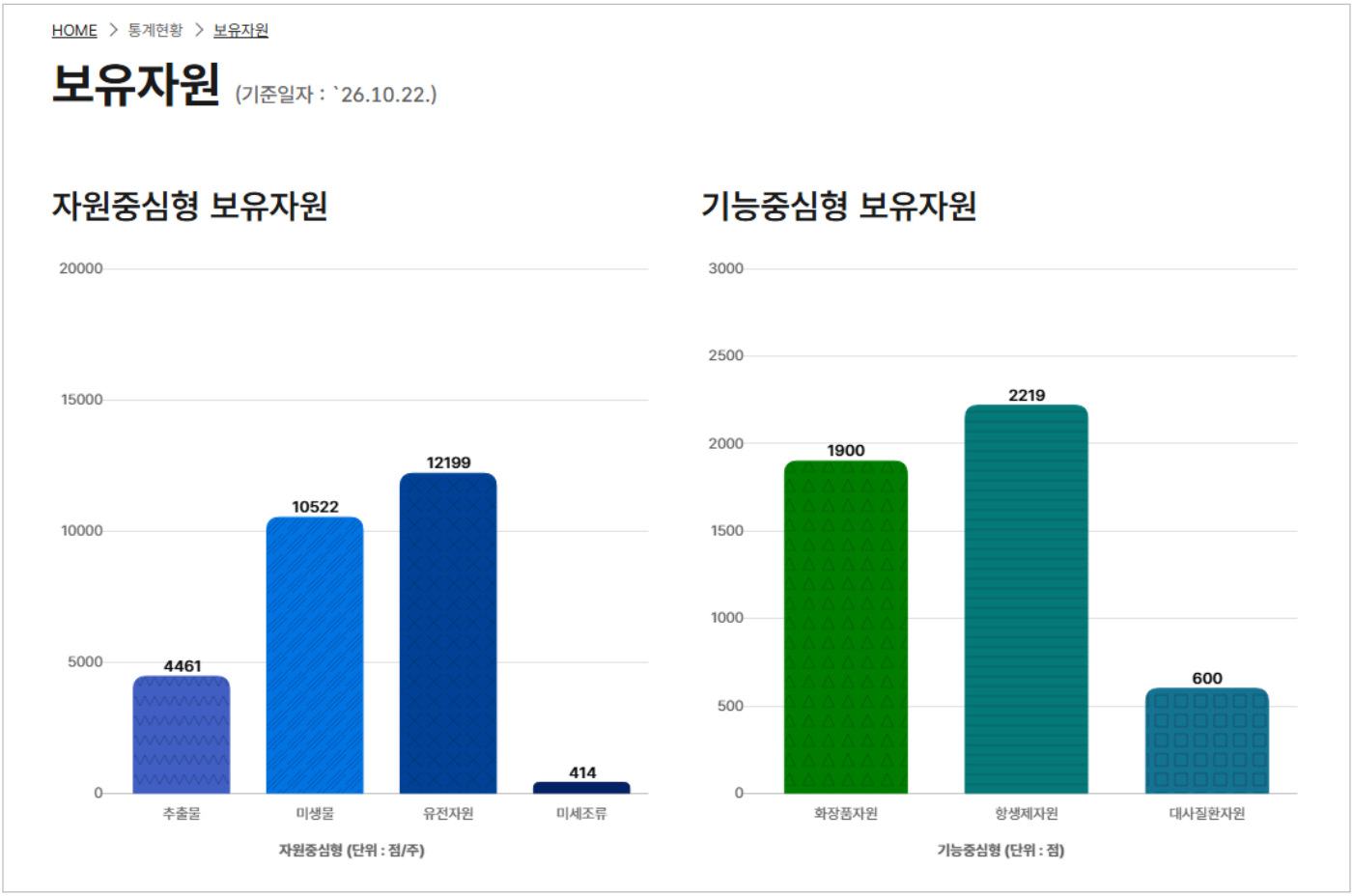
1. MBRIS 해양바이오뱅크 통계현황 (화면 캡처)

출처: MBRIS 통계현황(currentParcel.do, holdingRes.do) · 캡처 2026-10-22 (보유자원 수치는 10-12 수집값과 동일)

분양현황 — 뱅크별·산학연 누적 분양 (기준일자 2025.12.31.)



보유자원 (화면 기준일자 2026.10.22., 2026.10.12. 수집값과 수치 동일)



2. MBRIS 분양 안내와 성과현황 (화면 캡처)

출처: MBRIS 분양 안내(guide.do), 성과현황(performStatusBoard.do) · 캡처 2026-10-22 (0건은 10-21에도 확인)

분양 안내 — 분양승인기준(시행령 제17조)과 회당·연간 분양 한도

분양승인기준

• 해양수산생명자원의 확보·관리 및 이용 등에 관한 법률 시행령 제17조

1. 법 제20조제2항 각 호의 분양승인 제한사유에 해당하지 아니할 것
2. 해당 해양수산생명자원에 대한 권리자의 권리 보호에 지장이 없을 것
3. 해당 해양수산생명자원을 활용하기 위한 시설 및 장비 등을 갖추고 있을 것
4. 그 밖에 분양수량, 유전적 특성 또는 보전가치 등 해양수산부령으로 정하는 기준*을 충족 할 것

* 해양수산생명자원의 확보·관리 및 이용 등에 관한 법률 시행규칙 제8조

구분	유전자원	미생물·미세조류	추출물
회당	조직 : 최대 10점 (각 점당 최대 20mg 이하)	해양미생물 : 최대 20점 (각 점당 50ml 이하)	최대 20점 (각 점당 20ml 이하)
	핵산 : 최대 10점 (각 점당 최대 20μl 이하)	해양미세조류 : 최대 20점 (각 점당 100ml 이하)	
연간	조직 : 최대 200점	해양미생물 : 최대 50점	최대 100점
	핵산 : 최대 200점	해양미세조류 : 최대 50점	

* 화장품, 항생제 및 대사질환자원 분양은 추출물 기준과 동일하게 적용

분양 안내 — 유의사항(만족도 조사, 6개월 후 성과보고서 제출)

유의사항

• 분양받은 자원은 제3자에게 양도 및 재분양 할 수 없습니다.

• 신청하신 분양자원이 배양체인 경우, 일주일 이상의 배양과정을 거쳐 분양하게 되므로 일반 자원보다 더 많은 시일이 소요됨을 알려드립니다.

• 자원에 이상이 있는 경우 자원을 수령한 후 7일 이내에 분양 담당자에게 재분양을 신청할 수 있으며, 재분양 신청이 없는 경우 해당자원에 이상이 없는 것으로 간주합니다.

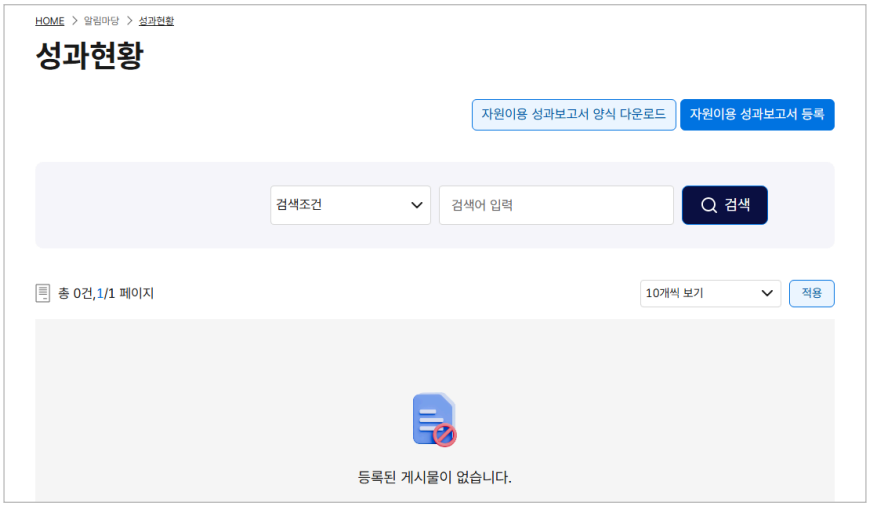
• 자원을 분양받은날로부터 1개월이 경과하면 해양바이오뱅크 이용에 대한 만족도 조사를 실시하오니 협조해주시기 바랍니다.

• 분양신청자는 분양 받은 자원을 이용한 **성과(학회 및 심포지엄 발표, 논문, 특허, 기술이전 등) 생산 시 출처**를 반드시 **표기**하여야 하며, 자원을 분양받은날로부터 6개월이 경과하면 **자원활용 성과보고서**와 함께 **증빙자료를 제출**하여야 합니다.

자원활용 성과보고서 등록하기

* 상기 사항에 어긋나는 경우, 분양이 제한될 수 있으니 유의사항을 숙지하여 주시기 바랍니다.

성과현황 게시판 — 등록된 게시물 없음(총 0건)



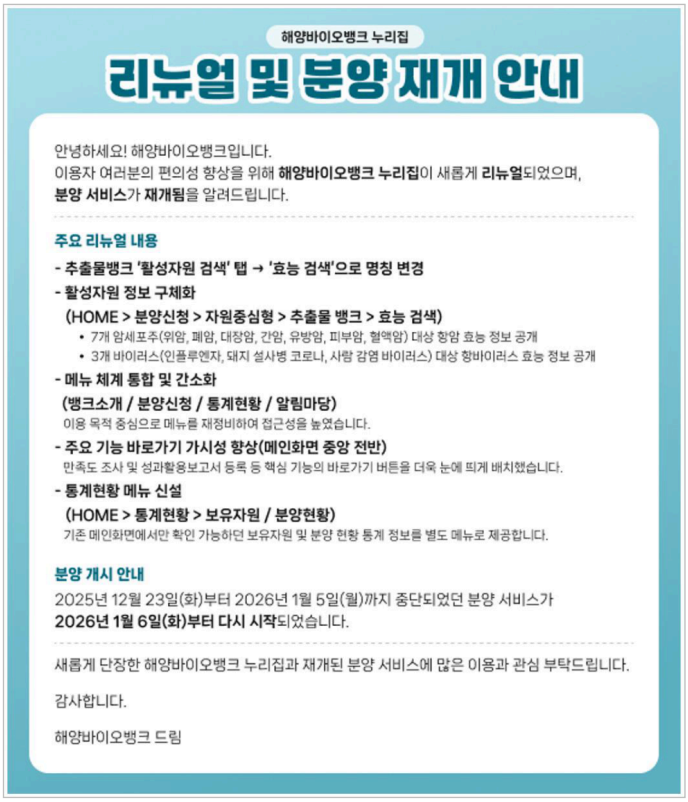
3. MBRIS 공지와 운영 전략 (화면 캡처)

출처: MBRIS 공지(2026.8.21., 2026.1.8.), 뱅크 소개 중 전략 · 공지 캡처 2026-10-12, 전략 캡처 2026-10-22

공지 2026.8.21. — 접수 월 3회, 수령 접수 마감 후 7~10 근무일



공지 2026.1.8. — 누리집 리뉴얼, 효능 검색으로 명칭 변경



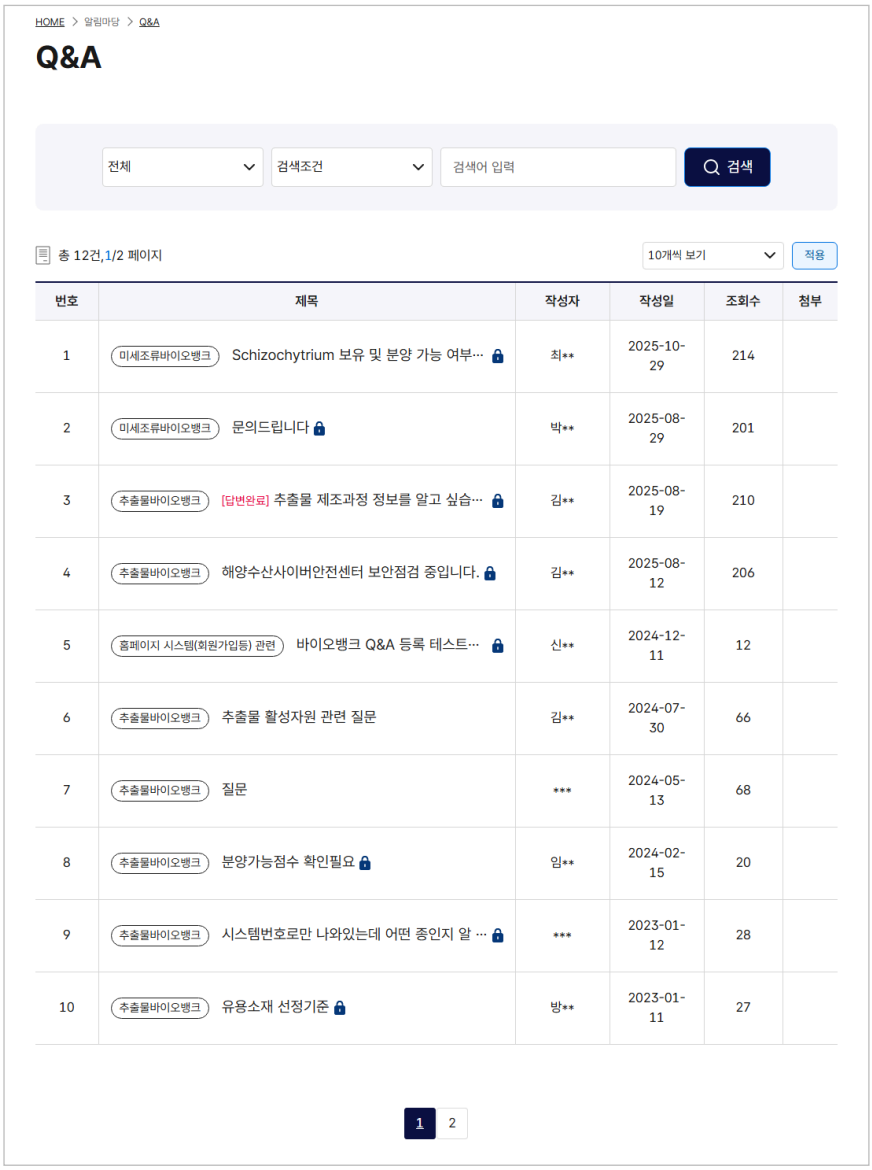
뱅크 소개 — 전략: 수요자 맞춤형 분양 서비스(수요자 기반 정보 구축 및 제공)



4. MBRIS Q&A 게시판과 효능 검색 화면 (화면 캡처)

출처: MBRIS Q&A(qna.do), 화장품자원 효능 검색 · Q&A 캡처 2026-10-22, 효능 검색 캡처 2026-10-12

Q&A 게시판 — 총 12건(1/2쪽). 시스템 테스트·보안 점검 문의 2건 제외 후 10건 분석



공개 게시글 본문 발췌

2024-05-13「질문」(추출물바이오뱅크)
활성자원만 따로 분류되어있는건 무슨 기준인건가요?
효능이 나와있는 자원만 분류해두신건지 궁금합니다.
그리고 활성자원에서 추출물이 어느 종에서 추출된건지 정보를 받고싶으면 어떻게 해야하나요?

2024-07-30「추출물 활성자원 관련 질문」
추출물 활성자원의 종을 확인하고 싶습니다.

화장품자원 효능 검색 화면

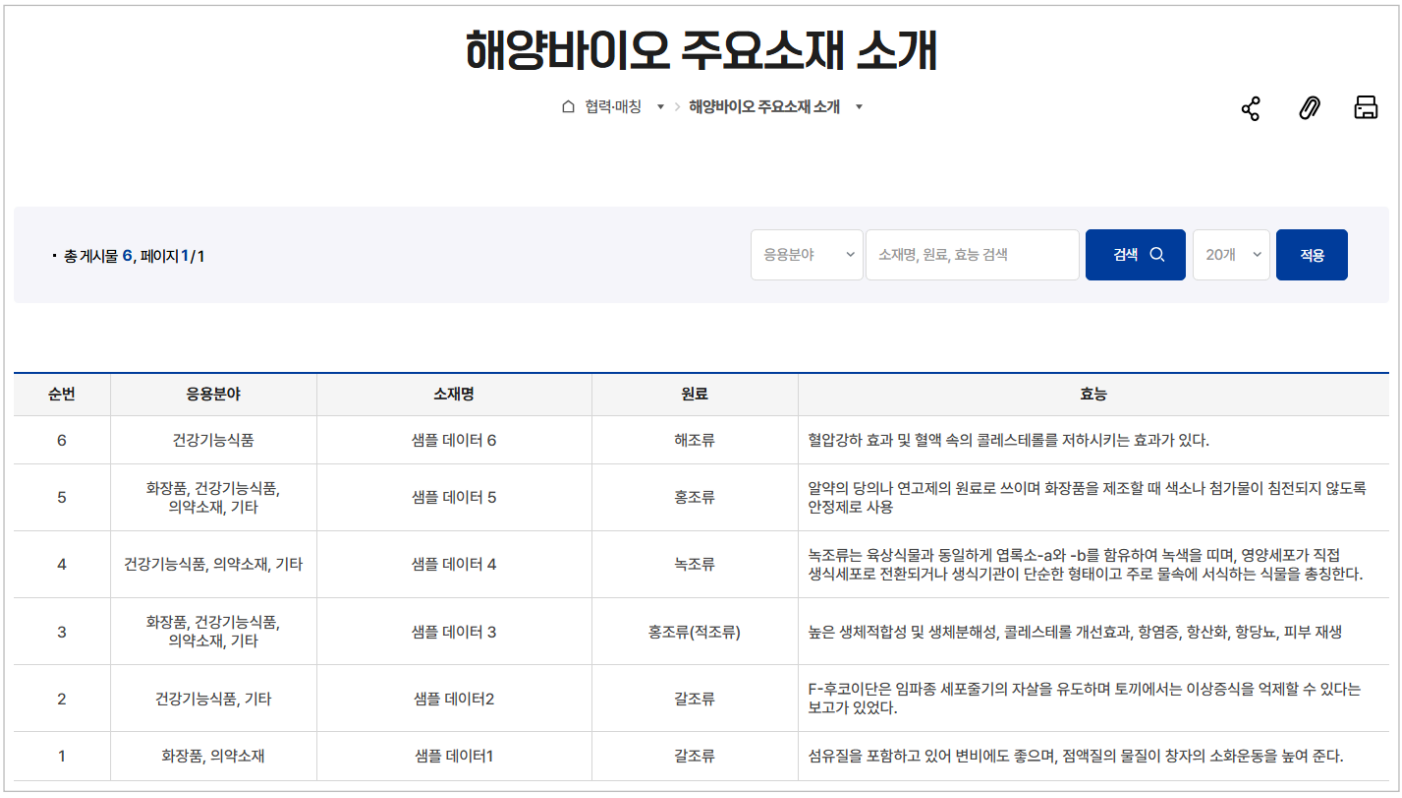
시험된 자원의 등급(A·B·C)만 조회



5. Mbio-Bridge 해양바이오 주요소재 소개와 기업수요 매칭 (화면 캡처)

출처: 국립해양생물자원관 Mbio-Bridge · 주요소재 캡처 2026-10-22, 기업수요 매칭 캡처 2026-10-12

해양바이오 주요소재 소개 — 총 6건, 소재명 모두 '샘플 데이터' (캡처 2026-10-22)



해양바이오 주요소재 소개

협력·매칭 › 해양바이오 주요소재 소개

· 총 게시물 6, 페이지 1/1

응용분야 | 소재명, 원료, 효능 검색 | 검색 | 20개 | 적용

순번	응용분야	소재명	원료	효능
6	건강기능식품	샘플 데이터 6	해조류	혈압강하 효과 및 혈액 속의 콜레스테롤를 저하시키는 효과가 있다.
5	화장품, 건강기능식품, 의약소재, 기타	샘플 데이터 5	홍조류	알약의 당의나 연고제의 원료로 쓰이며 화장품을 제조할 때 색소나 첨가물이 침전되지 않도록 안정제로 사용
4	건강기능식품, 의약소재, 기타	샘플 데이터 4	녹조류	녹조류는 육상식물과 동일하게 엽록소-a와 -b를 함유하여 녹색을 띠며, 영양세포가 직접 생식세포로 전환되거나 생식기관이 단순한 형태이고 주로 물속에 서식하는 식물을 총칭한다.
3	화장품, 건강기능식품, 의약소재, 기타	샘플 데이터 3	홍조류(적조류)	높은 생체적합성 및 생체분해성, 콜레스테롤 개선효과, 항염증, 항산화, 항당뇨, 피부 재생
2	건강기능식품, 기타	샘플 데이터2	갈조류	F-후코이단은 임파종 세포줄기의 자살을 유도하며 토끼에서는 이상증식을 억제할 수 있다는 보고가 있었다.
1	화장품, 의약소재	샘플 데이터1	갈조류	섬유질을 포함하고 있어 변비에도 좋으며, 점액질의 물질이 창자의 소화운동을 높여 준다.

기업수요 매칭 — 온라인 상담 신청 후 담당자가 확인하여 답변 또는 개별 연락



기업수요 매칭

협력·매칭 › 기업수요 매칭

해양바이오 매칭 서비스 온라인 상담 신청

- 해양생명자원에 대해 학술 및 산업화 연구를 위한 온라인 상담 서비스를 운영중이니 많은 이용 바랍니다.
- 기초 및 산업화 연구를 목적으로 하는 기업, 연구소, 교육기관 연구자 등 신청내용에 자유롭게 기재해주시면 담당자 확인 후 답변 또는 개별 연락을 드립니다.

6. 학술 문헌 (서지 정보와 초록 원문)

원문은 DOI로 확인 가능. 원문 사이트의 자동 접근 차단으로 서지 정보와 초록을 정리하여 수록함

참고자료 3. Phylogenies reveal predictive power of traditional medicine in bioprospecting.

Saslis-Lagoudakis CH, Savolainen V, Williamson EM, Forest F, Wagstaff SJ, Baral SR, Watson MF, Pendry CA, Hawkins JA. (2012). Phylogenies reveal predictive power of traditional medicine in bioprospecting. Proceedings of the National Academy of Sciences of the United States of America 109(39):15835- 15840. doi:10.1073/pnas.1202242109 PMCID: PMC3465383 · https://doi.org/10.1073/pnas.1202242109

초록 (원문)

There is controversy about whether traditional medicine can guide drug discovery, and investment in bioprospecting informed by ethnobotanical data has fluctuated. One view is that traditionally used medicinal plants are not necessarily efficacious and there are no robust methods for distinguishing those which are most likely to be bioactive when selecting species for further testing. Here, we reconstruct a genus-level molecular phylogenetic tree representing the 20,000 species found in the floras of three disparate biodiversity hotspots: Nepal, New Zealand, and the Cape of South Africa. Borrowing phylogenetic methods from community ecology, we reveal significant clustering of the 1,500 traditionally used species, and provide a direct measure of the relatedness of the three medicinal floras. We demonstrate shared phylogenetic patterns across the floras: related plants from these regions are used to treat medical conditions in the same therapeutic areas. This finding strongly indicates independent discovery of plant efficacy, an interpretation corroborated by the presence of a significantly greater proportion of known bioactive species in these plant groups than in random samples. We conclude that phylogenetic cross-cultural comparisons can focus screening efforts on a subset of traditionally used plants that are richer in bioactive compounds, and could revitalize the use of traditional knowledge in bioprospecting.

참고자료 4. BacDive in 2025: the core database for prokaryotic strain data.

Schober I, Koblitz J, Sardà Carbasse J, Ebeling C, Schmidt ML, Podstawka A, Gupta R, Ilangovan V, Chamanara J, Overmann J, Reimer LC. (2025). BacDive in 2025: the core database for prokaryotic strain data. Nucleic acids research 53(D1):D748-D756. doi:10.1093/nar/gkae959 PMCID: PMC11701647 · https://doi.org/10.1093/nar/gkae959

초록 (원문)

In 2025, the bacterial diversity database BacDive is the leading database for strain-level bacterial and archaeal information. It has been selected as an ELIXIR Core Data Resource as well as a Global Core Biodata Resource. Since its initial release more than ten years ago, BacDive (https://bacdive.dsmz.de) has grown tremendously in content and functionalities, and is a comprehensive resource covering the phenotypic diversity of prokaryotes with data on taxonomy, morphology, physiology, cultivation, and more. The current release (2023.2) contains 2.6 million data points on 97 334 strains, reflecting an increase by 52% since the previous publication in 2021. This remarkable growth can largely be attributed to the integration of the world-wide largest collection of Analytical Profile Index (API) test results, which are now fully integrated into the database and searchable. A novel BacDive knowledge graph provides powerful search options through a SPARQL endpoint, including the possibility for federated searches across multiple data sources. The high-quality data provided by BacDive is increasingly being used for the training of artificial intelligence models and resulting genome-based predictions with high confidence are now used to fill content gaps in the database.

7. 해외 사례와 인건비 산정 근거

NCI NPNPD(2026-10-12 조회) · 한국인공지능·소프트웨어산업협회「2026년 적용 SW기술자 평균임금 공표」(2025.12.19.)

NCI NPNPD — 천연물 분획 라이브러리, 23만 점 이상의 추출물 저장소를 연구 커뮤니티에 제공

The NCI Program for Natural Products Discovery (NPNPD)

The NPNPD was created to accelerate drug discovery. It is high-throughput amenable library comprising plant, marine and microbial biota from around the world. The Natural Products Branch (NPB) works with collaborators to provide natural product expertise in high-throughput screening. Depending on the level of collaboration, the NPB can provide faster analytical resources (isolation, structure elucidation, and resupply) to expedite translational pipelines.

The NCI is committed to the conservation of biological diversity and fair and equitable collaboration and compensation with the source countries.



Credit: Design by Christopher C. Thornburg

Abstract Summary

The US National Cancer Institute's (NCI) Natural Product Repository is one of the world's largest, most diverse collections of natural products containing over 230,000 unique extracts derived from plant, marine, and microbial organisms that have been collected from biodiverse regions throughout the world.

Importantly, this national resource is available to the research community for the screening of extracts and the isolation of bioactive natural products.

SW기술자 평균임금 — ① IT기획자 월 11,853,218원, ⑨ 응용SW개발자 월 7,754,124원 (담당자 연락처는 가림)

제목 2026년 적용 SW기술자 평균임금 공표 (조사년도 2025년)

등록일 2025-12-19 **담당자**

전화번호 **이메일**

(단위 : 원)

구 분	월평균임금 (M/M)	일평균임금 (M/D)	시간평균임금 (M/H)	포함직무
① IT 기획자	11,853,218	578,206	72,276	빅데이터기획자, 인공지능서비스 기획자
② IT 컨설턴트	10,707,960	522,340	65,292	정보보호컨설턴트
③ 업무분석가	9,740,667	475,154	59,394	
④ 데이터분석가	8,499,309	414,600	51,825	
⑤ IT PM	10,086,804	492,039	61,505	
⑥ IT 아키텍트	11,103,230	541,621	67,703	(SW,인프라스트럭처 ,데이터, 데이터베이스, 빅데이터,인공지능)아키텍트
⑦ UI/UX 기획 / 개발자	6,901,660	336,666	42,083	UI/UX 기획자, UI/UX 개발자
⑧ UI/UX 디자이너	5,159,246	251,671	31,459	
⑨ 응용 SW 개발자	7,754,124	378,250	47,281	빅데이터 개발자, 인공지능 SW 개발자

8. 언론 보도 (근거 문장 부분)

데일리안(2026.10.6.) · 아시아경제(2025.12.30.) · 푸드투데이(2026.10.10.) · 기자 연락처는 가림

데일리안「섬·연안 생물 유용물질 선별 빨라진다」

섬·연안 생물 유용물질 선별 빨라진다

섬·연안 생물자원에서 연구 가치가 높은 후보물질과 핵심 유전자를 선별하는 인공지능(AI) 기반 시스템을 구축했다.

국립호남권생물자원관(관장 박진영)은 생물융합연구부 김태수 선임연구원이 지난 7월 행정안전부 'AI챔피언 블루' 인증을 획득하고, 이를 바탕으로 연구지원형 AI 시스템을 개발했다고 밝혔다.

행안부 AI챔피언 제도는 국가·공공기관 실무자가 인공지능과 데이터로 행정 및 연구 문제를 해결하는 역량을 평가·인증하는 체계다.

이 중 '블루' 등급은 기획·설계부터 서비스 구현까지 실무 전 과정을 주도할 수 있는 전문 인재에게 수여된다.

김 선임연구원은 기관 내 데이터와 공개된 생명공학 데이터베이스를 연계해 유망 후보물질과 관여 유전자를 우선 선별하는 체계를 만들었다. 산업화 가능성이 높은 물질을 추리고 해당 물질 생산 유전자를 도출하는 과정을 지원해 연구 의사결정을 돕는 구조다.

아시아경제「AI로 바이오 소재 발굴·국산화 가속…국가 바이오데이터 700만건 구축」

AI로 바이오 소재 발굴·국산화 가속…국가 바이오데이터 700만건 구축

정부가 인공지능(AI)을 활용해 유용한 바이오 소재를 선제적으로 발굴하고, 수입 의존도가 높은 핵심 생명 소재의 단계적 국산화를 추진한다. 국가 연구·개발(R&D)과 공공·민간에 흩어진 바이오 데이터를 하나로 연결해 2030년까지 700만 건 이상의 국가 바이오 데이터를 구축하고, 바이오 AI 특화 모델 개발을 위한 전용 고성능 컴퓨팅 인프라도 단계적으로 확충한다.

대통령 직속 국가과학기술자문회의는 지난 23일부터 30일까지 제4회 심의회의를 서면으로 개최해 이 같은 내용을 담은 '제4차 국가생명연구자원 관리·활용 기본계획(2026~2030년)'과 '제1차 식품·의약품 규제과학혁신 기본계획(2026~2030년)'을 심의·의결

푸드투데이「[2026 국감] 해양바이오 연구과제 31건 중 상용화 7건…사업화 지원 시급」

[2026 국감] 해양바이오 연구과제 31건 중 상용화 7건…사업화 지원 시급

[푸드투데이 = 황인선기자] 해양바이오 소재와 활용기술 개발이 꾸준히 이뤄지고 있지만 실제 제품 출시로 이어지는 상용화 성과는 미미해 연구개발 이후 기업이 가장 큰 비용과 위험을 부담하는 스케일업·표준화·인허가 단계에 대한 공공 지원이 강화돼야 한다는 지적이다.

해양바이오뱅크의 소재 데이터베이스를 고도화해 기업 수요가 높은 소재를 체계적으로 관리하고, 미활용 특허의 기술이전뿐만 아니라 기술이전 이후 제품화까지 후속 지원하는 사후관리 체계를 구축해야 한다.

9. 공공데이터포털 공개 자료

국립해양생물자원관_해양생물 채집 공간정보 · 해양생물 화장품효능 (캡처 2026-10-12)

국립해양생물자원관_해양생물 채집 공간정보 — 학명·분류체계·채집 위치 (계통 정보 입력 자료)

CSV

국립해양생물자원관_해양생물 채집 공간정보

Quick AI Summary

해양생물 다양성 현황 파악

국립해양생물자원관이 제공하는 "국립해양생물자원관_해양생물 채집 공간정보" 데이터를 활용하면, 학명·국명·채집일자·위치 정보를 기반으로 국내 해양생물의 분포 현황을 시각화할 수 있습니다. 이를 통해 연안·심해·갯벌 등 다양한 서식환경별 종 풍부도를 정량적으로 분석할 수 있습니다. 연구기관 및 지방자치단체는 이러한 결과를 바탕으로 지역별 보전 우선순위를 설정하는 데 큰 도움을 받으실 수 있습니다.

해양환경 변화 모니터링

채집지의 위도·경도와 채집일자를 연계하면, 시간에 따른 서식지 이동 패턴을 추적할 수 있습니다. 특히 기후변화에 따른 온도·염도 변화가 특정 종의 서식범위에 미치는 영향을 정밀하게 분석할 수 있습니다. 해양수산·어촌 분야 담당자는 이 정보를 활용해 어업 정책 및 해양보호구역 설정에 과학적 근거를 제공하실 수 있습니다.

교육 및 시민과학 활용

학명·국명·분류체계가 포함된 데이터는 학교 교육 과정에서 해양생물 학습 자료로 활용하기에 적합합니다. 학생들은 실제 채집 위치와 환경 정보를 통해 현장감 있는 학습을 진행할 수 있으며, 시민과학 프로젝트에서도 참여자가 직접 관찰 기록을 데이터베이스와 비교할 수 있습니다. 이를 통해 해양생물에 대한 관심과 보전 의식을 고취시키는 데 기여하실 수 있습니다.

정책 수립 및 국제 협력

해양생물 채집 국가·해역 정보를 토대로 국내외 해양자원 관리 현황을 파악하고, 국제 협력 연구 과제에 필요한 기초 자료를 제공받을 수 있습니다. 관리부서인 국립해양생물자원관은 이 데이터를 기반으로 해양생물 자원 관리 계획을 체계화하고, 관련 법령 및 지침을 업데이트하는 데 활용하실 수 있습니다. 또한, 키워드인 "공간정보"와 "채집환경"을 중심으로 데이터 연계 분석을 수행하면 보다 정교한 정책 입안이 가능해집니다.

국립해양생물자원관_해양생물 화장품효능 — 200여 종 미백·주름개선·항산화 효능 정보

CSV

국립해양생물자원관_해양생물 화장품효능

Quick AI Summary

신제품 개발

국립해양생물자원관이 제공하는 해양생물 화장품효능 데이터는 200여 종의 해양생물에 대한 미백, 주름개선, 항산화 등 정량적 효능 정보를 포함하고 있어, 화장품 기업이 신제품 콘셉트를 설계할 때 원료 후보를 과학적으로 검증하는 근거 자료로 활용할 수 있습니다. 추출 방법·활성 성분·효능 지표가 체계적으로 정리돼 있어, 실험 설계 단계에서 목표 효능에 맞는 종을 빠르게 선별하고, 기존 연구와 비교 분석하여 차별화된 제품 라인을 구축할 수 있습니다.

기능성 원료 선별

연구기관은 데이터셋의 평균값·표준편차·유의수준 등을 활용해 특정 효능이 통계적으로 유의미한 해양생물을 식별하고, GIS 연계 분석을 통해 채취 지역별 원료 특성을 파악합니다. 이를 토대로 고효능 원료를 집중적으로 확보함으로써 원료 비용 절감과 품질 관리 효율성을 동시에 달성할 수 있습니다.

해양자원 관리 정책

공공기관은 데이터에 포함된 채취 지역 정보와 효능 지표를 기반으로 해양생물 자원의 지속 가능한 이용 방안을 수립합니다. 효능이 높은 종에 대한 과잉 채취 위험을 사전에 예측하고, 보호구역 지정이나 관리 계획을 과학적 근거에 따라 제안함으로써 해양 바이오·소재 산업의 장기적 성장과 환경 보전을 조화시킬 수 있습니다.

부록 A. 분석 방법과 결과 요약

본문 수치는 모두 공개 자료에서 분석 코드로 다시 계산한 값이며(재현 정보는 부록 B, 결과 표 전체는 부록 C, 코드는 부록 D).

1. 사용 데이터 (모두 공개 자료, 2026-10-12 수집)

- MBRIS 효능 등급 목록: 화장품 화면 6,015건(7개 시험) / 항균·항염·항암·항바이러스 화면 6,189건(18개 시험) / 대사질환 화면 600건(5개 시험). 일반 추출물 4,112건이 두 화면에 함께 표시되므로 자원번호 기준 중복을 제거한 고유 추출물은 8,808건이며 시험은 30종임
- MBRIS 통계현황: 뱅크별 보유 수(2026-10-12 기준)와 뱅크별 누적 분양·기관 유형별 분양(2025-12-31 기준)
- MBRIS 해양미생물 바이오뱅크 분양 목록 10,516주(누리집 보유자원 통계 10,522주와 6주 차이가 있으며 분석은 목록에서 조회된 10,516주 기준임) / 해양미세조류 바이오뱅크 목록 / 추출물 뱅크 4,461건 목록(분류체계 포함)
- 공공데이터포털 국립해양생물자원관 해양생물 채집 공간정보(학명·분류체계) / 해양생물 화장품효능
- 미생물·미세조류 속(genus)의 계통 정보: NCBI Taxonomy(문·강·목·과), 803개 속

2. 방법

- 입력: 생물 대분류(세균·균류·미세조류·기타)와 계통(문·강·목·과·속). 종명은 사용하지 않음
- 모델: 시험별 로지스틱 회귀(클래스 가중치 균형). 목표 변수는 해당 시험 A등급 여부. A등급이 15건 이상인 26개 시험만 평가
- 검증: 종 단위 5겹 교차검증. 같은 종의 추출물이 학습과 평가에 동시에 들어가지 않도록 종을 무작위로 5겹에 배정하며, 배정을 바꿔 10회 반복한 평균을 보고함
- 연간 분양 한도 모의 비교: 미생물 추출물만 대상으로 교차검증 예측값 상위 50건의 실제 A등급 수(반복 평균과 최소~최대)와 임의 선택 기대값(50 × A등급 비율)을 비교
- 표시 기준: 같은 속에서 시험된 미생물 종이 3종 이상이고 그 추출물이 10건 이상일 때만 예측 표시

3. 문제 진단 EDA

뱅크	개설	보유	누적 분양	1천 점당 연간 분양
미세조류	2020	414	194	78.1
추출물	2017	4,461	1,350	33.6
미생물	2018	10,522	453	5.4
대사질환	2025	600	2	3.3
유전자원	2019	12,199	253	3.0
화장품	2022	1,900	8	1.1
항생제	2023	2,219	5	0.8

산출: 누적 분양(2025년 말) ÷ 보유 수(2026년 10월) ÷ 운영 연수(개설 연도~2025). 보유 수는 현재값이므로 과거 보유가 적었던 뱅크의 활용도는 다소 낮게 계산됨. 효능 정보 유무와 분양률의 인과관계는 이 자료로 판단하지 않음(일반 추출물에도 효능 등급이 있어 경로가 겹침).

시험 30종의 A등급 비율 중앙값 3.9%, 최대 13.5%(지방축적(Oil Red O)). 살모넬라 0.03%, 대장균 0.05%. 누적 분양 2,265건 중 학교 69.1%(1,565건), 연구소 19.3%(437건), 기업 11.6%(263건).

이용자 체감 지표: 미생물 추출물의 황색포도상구균 항균 A등급 비율은 0.88%(n=2,384)이므로 연간 한도 50주를 임의로 고르면 기대 A등급은 0.44건, 1건 이상 얻을 확률은 1-(1-0.0088)^50 = 36%임. 예측 상위 50주의 10회 반복 적중은 평균 7.5건(15%), 최소 3건(6%)~최대 11건(22%)이며, 1건 이상 얻을 확률은 평균 기준 1-(1-0.15)^50 > 99.9%, 최소 기준 1-(1-0.06)^50 = 95.5%임. 첫 A등급까지 기대 신청 횟수는 회당 20점 기준 1/(1-(1-0.0088)^20) = 6.2회.

표시 가능 범위: 해양미생물 분양 목록 10,516주 중 효능 시험이 이루어진 종에 속하지 않는 균주 4,534주(분양 가능 4,451주). 이 중 표시 기준(같은 속 시험 종 3종 이상, 추출물 10건 이상)을 충족하는 균주 1,924주(42.4%).

인력·비용: 데이터 정비 2인월 + 화면·추천 개발 4인월(응용SW개발자 월 7,754,124원) + 사업관리 1인월(IT기획자 월 11,853,218원) = 직접인건비 58,377,962원. 2026년 적용 SW기술자 평균임금 기준.

4. 결과 요약

항목	결과
예측 정확도 (AUC)	황색포도상구균 0.913 · PGE2 0.903 · DPPH 0.924 · 평가한 26개 시험 평균 0.772 (시험별 값은 부록 C 표 1)
50주 모의 비교	황색포도상구균 임의 0.44건 → 예측 상위 평균 7.5건(3~11건) · ABTS는 4.14건 → 3.4건으로 개선 없음 (전체는 부록 C 표 3)

시험별 변동은 A등급 수가 적은 시험일수록 큼(예: 바실러스 세레우스).

방선균(Streptomyces): 추출물 62건 / 37종 중 황색포도상구균 A등급 비율 19.4% (미생물 추출물 평균 0.88%, 22배).

계통-효능 지도: 감태속·대황속(Ecklonia·Eisenia) 추출물 32건의 ABTS 항산화 A등급 비율 100%, 다시마목 전체(미역·다시마 포함) 83건은 39%. 사초과 PGE2 항염 72%(n=50), 그물바탕말목 65%(n=93). 전체 값은 부록 C 표 2에 수록.

5. 한계

- 효능 등급은 3단계이며 고정 농도의 시험관 내 결과임
- 같은 생물이 뱅크별로 다른 번호로 중복 등록되어 있어 검증은 종 단위로 분리함
- 같은 종도 채집 시기에 따라 등급이 달라짐. 예: 괭생이모자반(Sargassum horneri) 70% 에탄올 추출물의 PGE2 항염 등급이 2021년 5월 채집분은 C, 6월 채집분은 A (MBRIS 추출물 상세 정보 30건, 부록 B 자료 목록)
- 예측 상위 50주의 적중 수는 교차검증에서 종을 겹에 배정하는 방식에 따라 달라지므로 10회 반복의 평균과 범위로 표기함
- 같은 목이라도 속에 따라 효능이 달라 계통 단위를 세분하여 해석함(예: 다시마목 전체와 감태속·대황속의 차이)
- 암세포 시험 A등급은 일반 세포독성일 수 있어 서비스 근거에서 제외함

부록 B. 재현 정보와 데이터 목록

본문과 부록 A의 모든 수치는 아래 공개 자료와 부록 D의 코드로 다시 계산할 수 있음

□ 재현 방법

○ 환경: Python 3.12, pandas 3.0, numpy 2.4, scikit-learn 1.8, matplotlib 3.10 (그림에는 한글 글꼴 필요)

○ 순서: 01_eda_분양현황과A등급.py → 02_계통기반예측.py(약 10분) → 03_그림생성.py

- 경로는 코드 위치 기준 상대 경로이며 결과는 결과 폴더에 저장됨
- 난수는 종을 교차검증 겹에 배정할 때만 쓰며 시드 0~9로 고정함

○ 재현 확인: 코드와 자료를 새 폴더에 옮겨 처음부터 다시 실행한 결과가 제출 수치·그림과 같음을 확인함

□ 사용한 공개 자료 (모두 2026-10-12 수집)

파일	내용	출처	규모
efficacy_cosmetic.csv	화장품자원 효능 뱅크 표(시험 7종: 티로시나제·DOPA·콜라게나아제·엘라스타제·DPPH·ABTS·ROS)	MBRIS extractBeautyBank	6,015건
efficacy_antibac_inflam_cancer_virus.csv	항생제자원 효능 뱅크 표(시험 18종: 항균 6·항염 2·항암 7·항바이러스 3)	MBRIS extractAntiBac	6,189건
efficacy_metabolic.csv	대사질환자원 효능 뱅크 표(시험 5종: DPP4·α-글루코시다제·Oil Red O·중성지방·ACE)	MBRIS extractMtblsmDiss	600건
bank_microbe.csv	해양미생물 바이오뱅크 분양 목록(자원번호·학명·분양 상태)	MBRIS 미생물 뱅크	10,516주
bank_microalgae.csv	해양미세조류 바이오뱅크 목록	MBRIS 미세조류 뱅크	426주
bank_extract.csv	추출물 뱅크 목록(자원번호·분류체계·채집지·채집일)	MBRIS 추출물 뱅크	4,461건
collect_taxonomy.csv	학명별 분류체계(채집 공간정보에서 고유 학명만 추림)	공공데이터포털 해양생물 채집 공간정보	10,165종
ncbi_genus_lineage.json	미생물·미세조류 속의 문·강·목·과	NCBI Taxonomy E-utilities	803속
detail_sargassum_horneri.json	괭생이모자반 추출물 상세 정보(채집일·용매·시험별 등급)	MBRIS 추출물 상세 조회	30건

○ MBRIS 통계현황 수치(보유자원 2026.10.12., 분양현황 2025.12.31.)는 01번 코드에 상수로 적음

○ 효능 표의 열 순서는 MBRIS 표 머리글 순서와 같으며 시험 이름은 common.py의 ASSAYS에 있음

□ 분석 코드 구성 (전문은 부록 D)

파일	역할	분량
common.py	공통: 공개 자료 읽기, 학명 정리, 계통 정보 만들기	123줄
figstyle.py	그림 공통 서식	20줄
01_eda_분양현황과A등급.py	분양 활용도, 시험별 A등급 비율, 미생물 기준선, 방선균 비교, 그림 1	114줄
02_계통기반예측.py	시험별 AUC, 계통 깊이별 AUC, 50주 모의 비교, 계통-효능 지도, 미시험 미생물 현황	178줄
03_그림생성.py	그림 2·3 (02번 결과를 읽어 그림)	111줄
수집/scrape_mbris.py	MBRIS 효능 뱅크 표 수집(수집 방법 기록)	39줄
수집/ncbi_lineage.py	NCBI Taxonomy 계통 수집(수집 방법 기록)	76줄

□ 계통 정보 규칙

○ 출처 우선순위: 추출물 뱅크 목록의 분류체계 → 채집 공간정보 분류체계 → 같은 속의 분류체계. 미생물·미세조류는 NCBI Taxonomy

○ 계통-효능 지도의 어류·무척추동물은 문·강 수준 이름 목록으로 묶음(02번 코드의 FISH, INVERT)

부록 C. 결과 표

02번 코드 실행 결과. AUC는 종 단위 5겹 교차검증 10회 반복 평균이며, A등급이 15건 미만인 시험은 평가하지 않음(-)

□ 표 1. 시험 30종의 A등급 비율과 예측 정확도

효능 뱅크	시험	등급 있는 추출물	A등급 비율	AUC
화장품	티로시나제	3,243	2.68%	0.799
화장품	DOPA	3,244	0.99%	0.560
화장품	콜라게나아제	3,244	7.24%	0.830
화장품	엘라스타제	3,244	6.63%	0.830
화장품	DPPH	5,371	2.14%	0.924
화장품	ABTS	5,362	12.79%	0.798
화장품	ROS	5,251	3.22%	0.759
항균·항염·항암·항바이러스	황색포도상구균	5,775	5.13%	0.913
항균·항염·항암·항바이러스	바실러스 세레우스	5,775	0.81%	0.830
항균·항염·항암·항바이러스	장알균	4,728	1.57%	0.878
항균·항염·항암·항바이러스	대장균	5,774	0.05%	-
항균·항염·항암·항바이러스	살모넬라	5,774	0.03%	-
항균·항염·항암·항바이러스	녹농균	4,724	0.32%	0.758
항균·항염·항암·항바이러스	NO	5,609	12.23%	0.876
항균·항염·항암·항바이러스	PGE2	4,989	10.98%	0.903
항균·항염·항암·항바이러스	AGS위	5,848	5.04%	0.794
항균·항염·항암·항바이러스	A549폐	5,848	6.21%	0.804
항균·항염·항암·항바이러스	HCT116대장	5,848	6.87%	0.795
항균·항염·항암·항바이러스	HepG2간	5,848	4.09%	0.793
항균·항염·항암·항바이러스	MCF7유방	5,848	3.03%	0.854
항균·항염·항암·항바이러스	HT1080피부	5,848	6.79%	0.640
항균·항염·항암·항바이러스	HL60혈액	5,848	6.67%	0.767
항균·항염·항암·항바이러스	H1N1	5,710	3.61%	0.656
항균·항염·항암·항바이러스	코로나	5,711	2.87%	0.775
항균·항염·항암·항바이러스	A형간염	5,710	6.53%	0.562
대사질환	DPP4	600	13.33%	0.764
대사질환	α-글루코시다제	150	1.33%	-
대사질환	지방축적(Oil Red O)	600	13.50%	0.544
대사질환	중성지방	150	0.00%	-
대사질환	ACE	600	3.67%	0.659

○ A등급 비율 중앙값 3.9%, 평가한 26개 시험의 평균 AUC 0.772

□ 표 2. 계통-효능 지도 전체 값 (A등급 비율과 추출물 수)

계통	ABTS 항산화 평균 13%	DPPH 항산화 평균 2%	콜라게나아제 주름 평균 7%	포도상구균 항균 평균 5%	NO 항염 평균 12%	PGE2 항염 평균 11%	DPP4 혈당 평균 13%
감태속·대황속 (Ecklonia·Eisenia)	100% n=32	35% n=34	63% n=30	50% n=38	67% n=36	67% n=36	자료 부족
다시마목 전체 (미역·다시마 포함)	39% n=83	14% n=85	36% n=69	30% n=94	50% n=94	45% n=94	29% n=14
모자반속	35% n=234	6% n=234	29% n=161	32% n=278	55% n=249	51% n=249	27% n=30
그물바탕말목	24% n=91	1% n=92	23% n=65	25% n=100	58% n=93	65% n=93	10% n=10
갈파래목	3% n=59	2% n=59	0% n=55	0% n=71	52% n=60	20% n=60	자료 부족
사초과 (좀보리사초 등)	76% n=46	0% n=48	44% n=36	26% n=42	48% n=50	72% n=50	50% n=12
장미목 (해당화 등)	92% n=36	87% n=38	56% n=27	58% n=33	39% n=41	32% n=41	자료 부족
방선균 Streptomyces	18% n=65	0% n=65	0% n=58	19% n=62	3% n=62	0% n=62	자료 부족
홍조식물문	10% n=339	1% n=339	3% n=273	2% n=379	22% n=335	11% n=335	11% n=54
어류	6% n=377	0% n=377	0% n=313	0% n=438	1% n=358	0% n=358	6% n=34
무척추동물	5% n=1118	0% n=1119	1% n=815	0% n=1252	2% n=1170	0% n=1148	4% n=183

○ 추출물이 10건 미만인 칸은 '자료 부족'으로 표시함

□ 표 3. 연간 분양 한도 50주 모의 비교 (미생물 추출물)

목표 효능	임의 50건 기대 A	예측 상위 50건 실제 A (평균)	최소~최대 (10회)
황색포도상구균 항균	0.44	7.5	3~11
바실러스 세레우스 항균	0.25	9.6	6~11
NO 항염	1.41	6.0	2~11
PGE2 항염	1.30	6.2	1~7
ABTS 항산화	4.14	3.4	1~5

□ 표 4. 해양미생물 바이오뱅크 미시험 현황

분양 목록	효능 시험된 종에 속하지 않는 균주	그중 분양 가능	표시 기준 충족(같은 속 시험 종 3종 이상·추출물 10건 이상)
10,516주	4,534주	4,451주	1,924주 (42.4%)

부록 D. 분석 코드

부록 B의 재현 방법에 쓰인 코드 전문. 수집 코드 2개는 자료를 모은 방법을 기록하기 위한 것임

common.py · 공통: 공개 자료 읽기, 학명 정리, 계통 정보 만들기 · 123줄

```python
"""공통 모듈: 공개 자료를 읽어 효능 표(B·A·M)와 계통 정보를 만든다.

입력은 모두 ../데이터 의 공개 자료이다.
  efficacy_*.csv      MBRIS 효능 뱅크 화면의 표를 그대로 저장한 파일 (열 머리글 없음, 2026-10-12 수집)
  bank_*.csv          MBRIS 미생물·미세조류·추출물 뱅크 목록 (2026-10-12 수집)
  collect_taxonomy.csv  공공데이터포털 '해양생물 채집 공간정보'의 학명·분류체계 (고유 학명만 추림)
  ncbi_genus_lineage.json  NCBI Taxonomy에서 받은 미생물·미세조류 속(genus)의 문~과 계통
"""
import json
import os
import re
import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
HERE = os.path.dirname(os.path.abspath(__file__))
DATA = os.path.join(HERE, "..", "데이터")
RESULT = os.path.join(HERE, "..", "결과")
os.makedirs(RESULT, exist_ok=True)

# 효능 표의 열 순서 (MBRIS 표 머리글 순서와 동일)
ASSAYS = {
    "B": ["Tyros", "DOPA", "Collag", "Elas", "DPPH", "ABTS", "ROS"],
    "A": ["S.aureus", "B.cereus", "E.faecalis", "E.coli", "Salmonella", "P.aerug", "NO", "PGE2",
          "AGS위", "A549폐", "HCT116대장", "HepG2간", "MCF7유방", "HT1080피부", "HL60혈액",
          "H1N1", "코로나", "A형간염"],
    "M": ["DPP4", "aGlu", "ORO", "TG", "ACE"],
}
LABEL = {
    "Tyros": "티로시나제", "DOPA": "DOPA", "Collag": "콜라게나아제", "Elas": "엘라스타제", "DPPH": "DPPH", "ABTS": "ABTS",
    "ROS": "ROS", "S.aureus": "황색포도상구균", "B.cereus": "바실러스 세레우스", "E.faecalis": "장알균", "E.coli": "대장균",
    "Salmonella": "살모넬라", "P.aerug": "녹농균", "NO": "NO", "PGE2": "PGE2", "H1N1": "H1N1", "코로나": "코로나",
    "A형간염": "A형간염", "DPP4": "DPP4", "aGlu": "α-글루코시다제", "ORO": "지방축적(Oil Red O)", "TG": "중성지방", "ACE": "ACE",
}
FILES = {"B": "efficacy_cosmetic.csv", "A": "efficacy_antibac_inflam_cancer_virus.csv", "M": "efficacy_metabolic.csv"}
MICRO_TOKEN = ("BAC", "FUN", "ALG")


def read_csv(name, **kw):
    path = os.path.join(DATA, name)
    for enc in ("utf-8-sig", "cp949"):
        try:
            return pd.read_csv(path, encoding=enc, low_memory=False, **kw)
        except Exception:
            continue
    raise RuntimeError(path)


def binomial(s):
    """'Genus species 국명 -->' 같은 문자열에서 속명+종소명만 뽑는다."""
    s = re.sub(r"-->|\"|>", "", str(s))
    w = re.findall(r"[A-Za-z][A-Za-z.\-]*", s)
    return " ".join(w[:2])


class Taxonomy:
    """학명 → 계통(문.강.목.과.속) 토큰 목록."""

    def __init__(self):
        col = read_csv("collect_taxonomy.csv").dropna().drop_duplicates("학명").set_index("학명")["분류체계"]
        self.collect = col
        self.genus_chain = {}
        for sp, t in col.items():
            parts = [p for p in t.split(" > ") if p != sp]
            g = sp.split()[0]
            if g not in self.genus_chain or len(parts) > len(self.genus_chain[g]):
                self.genus_chain[g] = parts
        # 추출물 뱅크 목록의 분류체계(가장 상세) 우선
        self.bank = {}
        for v in read_csv("bank_extract.csv", header=None)[1].dropna():
```

```
            if "-->" not in v:
                continue
            sp = binomial(v.split("-->")[1]) if len(v.split("-->")) > 1 else ""
            parts = [re.sub(r'">$', "", x.strip()) for x in v.split("-->")[0].split(" > ")]
            if sp:
                self.bank[sp] = parts[:-1]
        self.ncbi = json.load(open(os.path.join(DATA, "ncbi_genus_lineage.json"), encoding="utf-8"))
        mb = read_csv("bank_microbe.csv", header=None)
        self.fungi = set(mb[mb[0].astype(str).str.contains("MABIK FU")][1].map(binomial))
        self.microbe_all = set(mb[1].map(binomial))
        self.algae = set(read_csv("bank_microalgae.csv", header=None)[1].map(binomial))
        self.micro = self.microbe_all | self.algae

    def lineage(self, sp):
        g = sp.split()[0]
        if sp in self.micro and self.ncbi.get(g):
            grp = "FUN" if sp in self.fungi else "ALG" if sp in self.algae else "BAC"
            return [grp] + self.ncbi[g] + ["g:" + g]
        if sp in self.bank:
            chain = self.bank[sp]
        elif sp in self.collect.index:
            chain = [p for p in str(self.collect[sp]).split(" > ") if p and p != sp]
        else:
            chain = self.genus_chain.get(g, [])
        return ["OTH"] + chain + ["g:" + g]


def load_tables():
    """B(화장품)·A(항균·항염·항암·항바이러스)·M(대사질환) 표. 자원번호 기준 중복 제거."""
    tax = Taxonomy()
    out = {}
    for key, fn in FILES.items():
        names = ASSAYS[key]
        d = read_csv(fn, header=None).iloc[:, : 2 + len(names)]
        d.columns = ["id", "raw"] + names
        d = d.drop_duplicates("id").copy()
        d["sp"] = d["raw"].map(binomial)
        d = d[d["sp"] != ""]
        d["L"] = d["sp"].map(tax.lineage)
        d["micro"] = d["sp"].isin(tax.micro)
        out[key] = d
    return out, tax


def graded(d, c):
    return d[d[c].isin(["A", "B", "C"])]


def has_token(L, key):
    """계통 토큰 목록에 key(예: 'Laminariales', 'g:Ecklonia')가 있는가. 토큰은 '라틴명 국명' 형식도 허용."""
    return any(t == key or t.split(" ")[0] == key for t in L)
```

figstyle.py · 그림 공통 서식 · 20줄

```
"""그림 공통 서식 (라이트 모드, 인쇄용)."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

SURF, INK, INK2, MUTED, GRID = "#fcfcfb", "#0b0b0b", "#52514e", "#8a8984", "#e6e5e0"
S1, S2, S3, S4 = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
NEUTRAL = "#b9b8b1"
SEQ = ["#fcfcfb", "#cde2fb", "#9ec5f4", "#6da7ec", "#3987e5", "#256abf", "#184f95", "#0d366b"]

from matplotlib import font_manager as _fm
_have = {f.name for f in _fm.fontManager.ttflist}
_FONT = next((n for n in ("Malgun Gothic", "AppleGothic", "NanumGothic", "Noto Sans CJK KR") if n in _have), "DejaVu Sans")

plt.rcParams.update({
    "font.family": _FONT, "axes.unicode_minus": False,
    "figure.facecolor": SURF, "axes.facecolor": SURF, "savefig.facecolor": SURF,
    "axes.edgecolor": GRID, "axes.labelcolor": INK2, "xtick.color": INK2, "ytick.color": INK2,
    "text.color": INK, "font.size": 12,
})
```

01_eda_분양현황과A등급.py · 분양 활용도, 시험별 A등급 비율, 미생물 기준선, 방선균 비교, 그림 1 · 114줄

```python
"""01. 탐색적 분석(EDA): 뱅크별 분양 활용도, 시험별 A등급 비율, 미생물 기준선, 방선균 비교.

실행: python 01_eda_분양현황과A등급.py      (결과는 ../결과 에 저장)
"""
import json
import os
import sys

import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
sys.stdout.reconfigure(encoding="utf-8")
from common import ASSAYS, LABEL, RESULT, graded, load_tables
import figstyle as fs
import matplotlib.pyplot as plt

T, tax = load_tables()
R = {}

# ── 1. 고유 추출물과 시험 수 ──────────────────────────
ids = set().union(*[set(d["id"]) for d in T.values()])
R["unique_extracts"] = len(ids)
rows = []
for key, d in T.items():
    for c in ASSAYS[key]:
        x = graded(d, c)
        rows.append({"table": key, "assay": c, "label": LABEL.get(c, c), "n_graded": len(x), "A_pct": (x[c] ==
"A").mean() * 100})
A_rate = pd.DataFrame(rows)
A_rate.to_csv(os.path.join(RESULT, "assay_A_rates.csv"), index=False, encoding="utf-8-sig")
R["n_assays"] = len(A_rate)
R["A_pct_median"] = round(float(A_rate["A_pct"].median()), 2)
R["A_pct_min"] = round(float(A_rate["A_pct"].min()), 2)
R["A_pct_max"] = [A_rate.loc[A_rate["A_pct"].idxmax(), "label"], round(float(A_rate["A_pct"].max()), 2)]
R["assays_A_le_0.1pct"] = A_rate[A_rate["A_pct"] <= 0.1]["label"].tolist()
print("고유 추출물", R["unique_extracts"], "| 시험", R["n_assays"], "| A등급 비율 중앙값", R["A_pct_median"], "최대",
R["A_pct_max"])
print("A등급 0.1% 이하 시험:", R["assays_A_le_0.1pct"])

# ── 2. 뱅크별 분양 활용도 (MBRIS 통계현황) ──────────────
# 보유: 보유자원 화면(2026-10-12 기준), 누적 분양: 분양현황 화면(2025-12-31 기준), 개설: 분양현황의 뱅크별 시작 연도
BANKS = [("미세조류", 2020, 414, 194), ("추출물", 2017, 4461, 1350), ("미생물", 2018, 10522, 453), ("대사질환", 2025, 600,
2),
         ("유전자원", 2019, 12199, 253), ("화장품", 2022, 1900, 8), ("항생제", 2023, 2219, 5)]
rate = [(n, h, c / (h / 1000) / (2025 - y + 1)) for n, y, h, c in BANKS]
R["bank_rate_per_1000_per_year"] = {n: round(v, 1) for n, _, v in rate}
SECTOR = {"기업": 263, "학교": 1565, "연구소": 437}  # 분양현황 '산·학·연 분양현황(누적)'
tot = sum(SECTOR.values())
R["sector_total"] = tot
R["sector_pct"] = {k: round(v / tot * 100, 1) for k, v in SECTOR.items()}
print("분양 활용도(1천 점·연):", R["bank_rate_per_1000_per_year"])
print("누적 분양", tot, R["sector_pct"])

# ── 3. 미생물 추출물 기준선과 방선균 ──────────────────────
A = T["A"]
mic = A[A["micro"]]
base = {}
for c in ["S.aureus", "B.cereus", "NO", "PGE2"]:
    p = (graded(mic, c)[c] == "A").mean()
    base[c] = {"A_pct": round(p * 100, 2), "expected_A_in_50": round(p * 50, 2), "n": int(len(graded(mic, c)))}
p = (graded(T["B"][T["B"]["micro"]], "ABTS")["ABTS"] == "A").mean()
base["ABTS"] = {"A_pct": round(p * 100, 2), "expected_A_in_50": round(p * 50, 2), "n": int(len(graded(T["B"][T["B"]
["micro"]], "ABTS")))}
R["microbe_baseline"] = base
p0 = base["S.aureus"]["A_pct"] / 100
R["P_at_least_1_A_random50"] = round(1 - (1 - p0) ** 50, 3)
R["first_A_expected_requests_20per"] = round(1 / (1 - (1 - p0) ** 20), 1)
st = A[A["sp"].str.startswith("Streptomyces")]
xs = graded(st, "S.aureus")
R["streptomyces"] = {"extracts": int(len(st)), "species": int(st["sp"].nunique()), "A_pct": round(float((xs["S.aureus"]
== "A").mean() * 100), 1),
                     "ratio_to_microbe_avg": round(float((xs["S.aureus"] == "A").mean() / p0), 1)}
print("미생물 기준선:", base)
print("임의 50주에서 A 1건 이상 확률", R["P_at_least_1_A_random50"], "| 첫 A까지 신청 횟수",
R["first_A_expected_requests_20per"])
print("방선균:", R["streptomyces"])

```

```python
# ── 4. 그림 1 ──────────────────────────────────────────
cat = {"미백·주름": ("B", ["Tyros", "DOPA", "Collag", "Elas"]), "항산화": ("B", ["DPPH", "ABTS", "ROS"]),
       "항균": ("A", ["S.aureus", "B.cereus", "E.faecalis", "E.coli", "Salmonella", "P.aerug"]), "항염": ("A", ["NO", "PGE2"]),
       "항암": ("A", ["AGS위", "A549폐", "HCT116대장", "HepG2간", "MCF7유방", "HT1080피부", "HL60혈액"]),
       "항바이러스": ("A", ["H1N1", "코로나", "A형간염"]), "대사질환": ("M", ["DPP4", "aGlu", "ORO", "TG", "ACE"])}
pts = [(k, c, (graded(T[t], c)[c] == "A").mean() * 100) for k, (t, cs) in cat.items() for c in cs]
med = float(np.median([v for _, _, v in pts]))
S1, S2, SURF, INK, INK2, MUTED, GRID, NEUTRAL = fs.S1, fs.S2, fs.SURF, fs.INK, fs.INK2, fs.MUTED, fs.GRID, fs.NEUTRAL
fig, (a1, a2) = plt.subplots(1, 2, figsize=(8.6, 3.3), dpi=220, gridspec_kw={"width_ratios": [1.05, 1]})
names = [f"{n}  (보유 {h:,})" for n, h, _ in rate]
hl = {"미생물", "유전자원"}
yy = np.arange(len(rate))[::-1]
for yv, (n, h, v) in zip(yy, rate):
    a1.barh(yv, v, height=0.62, color=S1 if n in hl else NEUTRAL, zorder=3)
    a1.text(v + 1.2, yv, f"{v:.1f}", va="center", fontsize=9.6, color=INK if n in hl else INK2, fontweight="bold" if n in hl else None)
a1.set_yticks(yy, names, fontsize=9.6); a1.set_xlim(0, 90); a1.tick_params(length=0)
a1.grid(axis="x", color=GRID, lw=0.8); a1.set_axisbelow(True)
for s_ in ("top", "right", "left"): a1.spines[s_].set_visible(False)
a1.set_xlabel("보유 1천 점당 연간 분양 건수", fontsize=10.2)
a1.set_title("(가) 뱅크별 분양 활용도", loc="left", fontsize=12.0, fontweight="bold")
a1.text(43, 2.0, "보유 자원이 가장 많은\n미생물·유전자원은\n활용도가 낮음", fontsize=9.6, color=S1, va="center")
order = list(cat.keys())
for i, k in enumerate(order):
    xs_ = [v for kk, _, v in pts if kk == k]
    a2.scatter(xs_, [len(order) - 1 - i] * len(xs_), s=34, color=S1, edgecolor=SURF, linewidth=1.2, zorder=3, alpha=0.9)
a2.axvline(med, color=S2, lw=1.4, ls=(0, (4, 3)), zorder=2)
a2.text(med + 0.3, -0.55, f"중앙값 {med:.1f}%", color=S2, fontsize=9.6, fontweight="bold", va="bottom")
a2.set_yticks(range(len(order)), order[::-1], fontsize=10.2); a2.tick_params(length=0)
a2.set_xlim(-0.6, 15); a2.set_ylim(-0.75, len(order) - 0.3)
a2.grid(axis="x", color=GRID, lw=0.8); a2.set_axisbelow(True)
for s_ in ("top", "right", "left"): a2.spines[s_].set_visible(False)
a2.set_xlabel("시험별 A등급 비율 (%)", fontsize=10.2)
a2.set_title("(나) 효능 시험 30종의 A등급 비율", loc="left", fontsize=12.0, fontweight="bold")
yk = len(order) - 1 - order.index("항균")
a2.text(0.0, yk - 0.42, "살모넬라 0.0%·대장균 0.1%", fontsize=8.2, color=INK2, va="center", zorder=5, bbox=dict(boxstyle="square,pad=0.1", fc=SURF, ec="none"))
fig.suptitle("그림 1. 분양 활용 현황과 A등급 자원의 희소성", x=0.01, ha="left", fontsize=14.4, fontweight="bold")
fig.text(0.01, 0.01, "(가) 누적 분양(2025년 말) ÷ 보유 수(2026년 10월) ÷ 운영 연수. 자료: MBRIS 통계현황  (나) 효능 등급 추출물 8,808건, 점 하나가 시험 1종", fontsize=8.4, color=MUTED)
fig.tight_layout(rect=(0, 0.035, 1, 0.965))
fig.savefig(os.path.join(RESULT, "그림1_EDA_분양현황과A등급.png")); plt.close(fig)
R["fig1_median_A"] = round(med, 1)
json.dump(R, open(os.path.join(RESULT, "eda.json"), "w", encoding="utf-8"), ensure_ascii=False, indent=1)
print("저장 완료 →", RESULT)
```

02_계통기반예측.py · 시험별 AUC, 계통 깊이별 AUC, 50주 모의 비교, 계통-효능 지도, 미시험 미생물 현황 · 178줄

```python
"""02. 계통 기반 효능 예측: 계통-효능 지도, 예측 정확도(AUC), 연간 분양 한도 모의 비교, 미시험 미생물 현황.

방법
- 입력: 생물 대분류(세균·균류·미세조류·기타)와 계통(문·강·목·과·속). 종명 자체는 사용하지 않는다.
- 모델: 시험별 로지스틱 회귀(클래스 가중치 균형). 목표 변수는 해당 시험의 A등급 여부.
- 검증: 같은 종의 추출물이 학습과 평가에 함께 들어가지 않도록 종 단위로 나누는 5겹 교차검증(GroupKFold).
- 모의 비교: 미생물 추출물만 대상으로 교차검증 예측값 상위 50건의 실제 A등급 수와 임의 50건의 기대값을 비교.

실행: python 02_계통기반예측.py      (결과는 ../결과/model_summary.json 등에 저장, 약 10분 소요)
그림은 03_그림생성.py 가 이 결과 파일을 읽어 그린다.
"""
import json
import os
import sys

import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
sys.stdout.reconfigure(encoding="utf-8")
from common import ASSAYS, DATA, LABEL, RESULT, binomial, graded, has_token, load_tables

T, tax = load_tables()
B, A, M = T["B"], T["A"], T["M"]
R = {"ncbi_genera": len(tax.ncbi)}


# ── 교차검증 ──────────────────────────────────────────
```

```python
def depth_tokens(L, lvl):
    """계통 깊이 0=대분류만, 1=문~목, 2=과, 3=속까지."""
    if lvl == 0:
        return [L[0]]
    core = [t for t in L if not t.startswith("g:")]
    if lvl == 1:
        return core[:max(1, len(core) - 2)]
    if lvl == 2:
        return core[:max(1, len(core) - 1)]
    return L


REPS = 10  # 종을 5겹에 배정하는 방식을 바꿔 가며 되풀이하는 횟수 (한 번의 분할에 따른 우연을 줄이기 위함)


def fold_ids(groups, seed):
    """같은 종은 같은 겹에 들어가도록 종 단위로 5겹에 무작위 배정."""
    u = np.unique(groups)
    perm = np.random.RandomState(seed).permutation(len(u))
    fid = dict(zip(u, perm % 5))
    return np.array([fid[g] for g in groups])


def cv_predict(d, c, lvl=3, seed=0):
    x = graded(d, c)
    X = CountVectorizer(analyzer=lambda t: t).fit_transform(x["L"].apply(lambda L: depth_tokens(L, lvl)))
    y = (x[c] == "A").values.astype(int)
    f = fold_ids(x["sp"].values, seed)
    p = np.zeros(len(y))
    for k in range(5):
        tr, te = np.where(f != k)[0], np.where(f == k)[0]
        p[te] = LogisticRegression(max_iter=3000, class_weight="balanced").fit(X[tr], y[tr]).predict_proba(X[te])[:, 1]
    return x, y, p


def mean_auc(d, c, lvl=3):
    v = []
    for s in range(REPS):
        _, y, p = cv_predict(d, c, lvl, s)
        v.append(roc_auc_score(y, p))
    return round(float(np.mean(v)), 3), round(float(np.std(v)), 3)


# ── 1. 시험별 AUC (분할 10회 평균) ──────────────────────
auc, auc_sd = {}, {}
for key, d in T.items():
    for c in ASSAYS[key]:
        x = graded(d, c)
        if (x[c] == "A").sum() < 15:
            continue
        auc[c], auc_sd[c] = mean_auc(d, c)
R["auc"] = auc
R["auc_sd"] = auc_sd
R["reps"] = REPS
R["auc_mean"] = round(float(np.mean(list(auc.values()))), 3)
R["n_assays"] = len(auc)
pd.DataFrame([{"assay": c, "label": LABEL.get(c, c), "AUC": v} for c, v in sorted(auc.items(), key=lambda kv: -
kv[1])]).to_csv(
    os.path.join(RESULT, "auc_by_assay.csv"), index=False, encoding="utf-8-sig")
print(f"AUC 산출 시험 {R['n_assays']}개, 평균 {R['auc_mean']}")
print({LABEL.get(c, c): v for c, v in sorted(auc.items(), key=lambda kv: -kv[1])})

# ── 2. 계통 깊이별 AUC ──────────────────────────────────
series = [("황색포도상구균 항균", A, "S.aureus"), ("PGE2 항염", A, "PGE2"), ("콜라게나아제 주름", B, "Collag"), ("인플루엔자
항바이러스", A, "H1N1")]
CURVES = {}
for name, d, c in series:
    CURVES[c] = [mean_auc(d, c, lvl)[0] for lvl in range(4)]
R["auc_by_depth"] = CURVES
print("깊이별 AUC:", CURVES)

# ── 3. 연간 분양 한도 50주 모의 비교 ────────────────────
res = []
for name, d, c in [("황색포도상구균 항균", A, "S.aureus"), ("바실러스 세레우스 항균", A, "B.cereus"), ("NO 항염", A, "NO"),
                   ("PGE2 항염", A, "PGE2"), ("ABTS 항산화", B, "ABTS")]:
    hits = []
    for s in range(REPS):
        x, y, p = cv_predict(d, c, 3, s)
        m = x["micro"].values
        pm, ym = p[m], y[m]
        top = np.argsort(-pm, kind="stable")[:50]
        hits.append(int(ym[top].sum()))
```

```
    res.append((name, round(float(50 * ym.mean()), 3), round(float(np.mean(hits)), 1), int(min(hits)), int(max(hits))))
R["sim50"] = [list(r) for r in res]
print("50주 모의(이름, 임의 기대, 예측 평균, 최소, 최대):", res)
pd.DataFrame(res, columns=["test", "random_expected_A", "predicted_top50_A_mean", "min", "max"]).to_csv(
    os.path.join(RESULT, "sim50.csv"), index=False, encoding="utf-8-sig")
hit = res[0][2] / 50
R["S.aureus_top50_hit_rate"] = hit
R["P_at_least_1_A_predicted50"] = round(1 - (1 - hit) ** 50, 4)

# ── 4. 방선균 ──────────────────────────────────────
st = A[A["sp"].str.startswith("Streptomyces")]
xs = graded(st, "S.aureus")
mic = graded(A[A["micro"]], "S.aureus")
R["strep"] = {"extracts": int(len(st)), "species": int(st["sp"].nunique()), "A_rate": round(float((xs["S.aureus"] ==
"A").mean()), 3),
              "micro_base": round(float((mic["S.aureus"] == "A").mean()), 4)}
print("방선균:", R["strep"])

# ── 5. 계통-효능 지도 ──────────────────────────────
FISH = ["Teleostei", "Chondrichthyes", "Elasmobranchii", "Actinopterygii", "Holocephali"]
INVERT = ["Mollusca", "Arthropoda", "Echinodermata", "Cnidaria", "Porifera", "Annelida", "Bryozoa", "Nemertea",
"Platyhelminthes", "Sipuncula"]
rows = [("감태속·대황속\n(Ecklonia·Eisenia)", ["g:Ecklonia", "g:Eisenia"]), ("다시마목 전체\n(미역·다시마 포함)",
["Laminariales"]),
        ("모자반속", ["g:Sargassum"]), ("그물바탕말목", ["Dictyotales"]), ("갈파래목", ["Ulvales"]),
        ("사초과\n(좀보리사초 등)", ["Cyperaceae"]), ("장미목\n(해당화 등)", ["Rosales"]),
        ("방선균\nStreptomyces", ["g:Streptomyces"]), ("홍조식물문", ["Rhodophyta"]), ("어류", FISH), ("무척추동물",
INVERT)]
cols = [("ABTS\n항산화", B, "ABTS"), ("DPPH\n항산화", B, "DPPH"), ("콜라게나아제\n주름", B, "Collag"),
        ("포도상구균\n항균", A, "S.aureus"), ("NO\n항염", A, "NO"), ("PGE2\n항염", A, "PGE2"), ("DPP4\n혈당", M, "DPP4")]
mat = np.full((len(rows), len(cols)), np.nan)
nmat = np.zeros_like(mat)
for i, (_, keys) in enumerate(rows):
    for j, (_, d, c) in enumerate(cols):
        x = graded(d, c)
        x = x[x["L"].apply(lambda L: any(has_token(L, k) for k in keys))]
        if len(x) >= 10:
            mat[i, j] = (x[c] == "A").mean() * 100
            nmat[i, j] = len(x)
avg = [(graded(d, c)[c] == "A").mean() * 100 for _, d, c in cols]
rlab = [r[0].replace("\n", " ") for r in rows]
clab = [c[0].replace("\n", " ") for c in cols]
pd.DataFrame(mat, index=rlab, columns=clab).round(1).to_csv(os.path.join(RESULT, "lineage_heatmap_A_pct.csv"),
encoding="utf-8-sig")
pd.DataFrame(nmat, index=rlab, columns=clab).astype(int).to_csv(os.path.join(RESULT, "lineage_heatmap_n.csv"),
encoding="utf-8-sig")
R["heatmap"] = {"rows": rlab, "cols": clab, "A_pct": [[None if np.isnan(v) else round(float(v), 1) for v in r] for r in
mat],
                "n": nmat.astype(int).tolist(), "avg": [round(float(a), 1) for a in avg]}
print(pd.DataFrame(mat, index=rlab, columns=clab).round(0).to_string())
print("평균 A%:", [round(float(a), 1) for a in avg])

# ── 6. 미시험 미생물과 표시 가능 범위 ──────────────
mb = pd.read_csv(os.path.join(DATA, "bank_microbe.csv"), header=None, encoding="utf-8-sig")
mb["sp"] = mb[1].map(binomial)
mb["avail"] = mb[6].astype(str).str.contains("분양가능")
tested_sp = set(pd.concat([B["sp"], A["sp"], M["sp"]]))
mb["untested"] = ~mb["sp"].isin(tested_sp)
ge = pd.concat([B[["sp", "micro", "id"]], A[["sp", "micro", "id"]], M[["sp", "micro", "id"]]])
ge = ge[ge["micro"]].drop_duplicates("id")
ge["g"] = ge["sp"].str.split().str[0]
gsp = ge.groupby("g")["sp"].nunique()
gcnt = ge.groupby("g")["id"].nunique()
ok_genus = set(gsp[gsp >= 3].index) & set(gcnt[gcnt >= 10].index)
mb["g"] = mb["sp"].str.split().str[0]
mb["display"] = mb["untested"] & mb["g"].isin(ok_genus)
R["microbe_bank"] = {"total": int(len(mb)), "untested": int(mb["untested"].sum()),
                     "untested_available": int((mb["untested"] & mb["avail"]).sum()),
                     "display_ok": int(mb["display"].sum()),
                     "display_ok_pct_of_untested": round(float(mb["display"].sum() / mb["untested"].sum() * 100), 1),
                     "status_counts": {k: int(v) for k, v in mb[6].astype(str).str.extract(r"(분양가능|분양준비중|분양불
가)")[0].value_counts().items()}}
print("미생물 뱅크:", R["microbe_bank"])
json.dump(R, open(os.path.join(RESULT, "model_summary.json"), "w", encoding="utf-8"), ensure_ascii=False, indent=1)
print("저장 완료 →", RESULT)
```

03_그림생성.py · 그림 2-3 (02번 결과를 읽어 그림) · 111줄

```
"""03. 그림 2-3 생성: 02번 스크립트가 저장한 ../결과/model_summary.json 을 읽어 그린다 (계산 없음).

```

```python
실행: python 03_그림생성.py
"""
import json
import os
import sys

import numpy as np
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.patches import Patch

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
sys.stdout.reconfigure(encoding="utf-8")
from common import RESULT
import figstyle as fs
import matplotlib.pyplot as plt

SURF, INK, INK2, MUTED, GRID = fs.SURF, fs.INK, fs.INK2, fs.MUTED, fs.GRID
S1, S2, S3, NEUTRAL = fs.S1, fs.S2, fs.S3, fs.NEUTRAL
R = json.load(open(os.path.join(RESULT, "model_summary.json"), encoding="utf-8"))
HM = R["heatmap"]
ROW_FMT = ["감태속·대황속\n(Ecklonia·Eisenia)", "다시마목 전체\n(미역·다시마 포함)", "모자반속", "그물바탕말목", "갈파래목",
           "사초과\n(좀보리사초 등)", "장미목\n(해당화 등)", "방선균\nStreptomyces", "홍조식물문", "어류", "무척추동물"]
COL_FMT = ["ABTS\n항산화", "DPPH\n항산화", "콜라게나아제\n주름", "포도상구균\n항균", "NO\n항염", "PGE2\n항염", "DPP4\n혈당"]
assert [r.replace("\n", " ") for r in ROW_FMT] == HM["rows"] and [c.replace("\n", " ") for c in COL_FMT] == HM["cols"]
# 저장값은 소수 첫째 자리로 반올림돼 있으므로 건수(n)로 A등급 건수를 되살려 정확한 비율을 쓴다 (이중 반올림 방지)
mat = np.array([[np.nan if v is None else round(v * n / 100) / n * 100 for v, n in zip(row, nrow)]
                for row, nrow in zip(HM["A_pct"], HM["n"])], dtype=float)
avg = HM["avg"]

# ── 그림 2: 계통-효능 지도 ──────────────────────────────
fig, ax = plt.subplots(figsize=(8.6, 6.2), dpi=220)
cmap = LinearSegmentedColormap.from_list("seq", fs.SEQ)
im = ax.imshow(np.nan_to_num(mat, nan=-1), cmap=cmap, vmin=0, vmax=100, aspect="auto")
for i in range(len(ROW_FMT)):
    for j in range(len(COL_FMT)):
        v = mat[i, j]
        if np.isnan(v):
            ax.add_patch(plt.Rectangle((j - .5, i - .5), 1, 1, color="#f0efec", zorder=2))
            ax.text(j, i, "자료\n부족", ha="center", va="center", fontsize=8.4, color=MUTED, zorder=3)
        else:
            ax.text(j, i, f"{v:.0f}%", ha="center", va="center", fontsize=11.0, fontweight="bold",
                    color="white" if v >= 45 else INK, zorder=3)
for k in range(len(ROW_FMT) + 1):
    ax.axhline(k - .5, color=SURF, lw=2, zorder=4)
for k in range(len(COL_FMT) + 1):
    ax.axvline(k - .5, color=SURF, lw=2, zorder=4)
ax.set_yticks(range(len(ROW_FMT)), ROW_FMT, fontsize=9.6)
ax.set_xticks(range(len(COL_FMT)), [f"{c}\n(평균 {a:.0f}%)" for c, a in zip(COL_FMT, avg)], fontsize=9.6)
ax.xaxis.tick_top()
ax.tick_params(length=0)
for s in ax.spines.values():
    s.set_visible(False)
cb = fig.colorbar(im, ax=ax, fraction=0.035, pad=0.02)
cb.outline.set_visible(False)
cb.set_label("A등급 비율 (%)", color=INK2)
cb.ax.tick_params(colors=INK2, length=0)
fig.text(0.01, 0.015, "자료: MBRIS 효능 등급 추출물 8,808건(중복 제거) 분석. 계통별 추출물 10건 미만은 '자료 부족'으로 표시.",
fontsize=9.0, color=MUTED)
fig.suptitle("그림 2. 계통별 효능 A등급 비율 (계통-효능 지도)", x=0.01, ha="left", fontsize=14.4, fontweight="bold")
fig.tight_layout(rect=(0, 0.03, 1, 0.96))
fig.savefig(os.path.join(RESULT, "그림2_계통효능지도.png"))
plt.close(fig)

# ── 그림 3: 예측 정확도 + 50주 모의 비교 ──────────────────
CURVES, res = R["auc_by_depth"], R["sim50"]
fig, (b1, b2) = plt.subplots(1, 2, figsize=(8.6, 3.25), dpi=220, gridspec_kw={"width_ratios": [1, 1.1]})
lab = {"S.aureus": ("항균(포도상구균)", S1, 0.024), "PGE2": ("항염(PGE2)", S2, -0.024), "Collag": ("주름(콜라게나아제)",
S3, 0),
       "H1N1": ("항바이러스(H1N1)", NEUTRAL, 0)}
b1.axhline(0.5, color=MUTED, lw=1, ls=(0, (4, 3)))
b1.text(1.75, 0.512, "무작위 0.5", fontsize=9.0, color=MUTED, va="bottom")
for c, v in CURVES.items():
    nm, col, off = lab[c]
    b1.plot(range(4), v, color=col, lw=2, marker="o", ms=6, mec=SURF, mew=1.5, zorder=3)
    b1.text(3.12, v[-1] + off, f"{nm} {v[-1]:.2f}", va="center", fontsize=9.0, color=INK2)
b1.set_xticks(range(4), ["대분류", "문~목", "과", "속"], fontsize=9.6)
b1.set_xlim(-0.25, 5.6)
b1.set_ylim(0.45, 1.0)
b1.set_ylabel("AUC (10회 평균)", fontsize=10.2)
b1.tick_params(labelsize=8)
b1.grid(axis="y", color=GRID, lw=0.8)
b1.set_axisbelow(True)
```

```
for s_ in ("top", "right"):
    b1.spines[s_].set_visible(False)
b1.set_title("(가) 계통 정보 깊이별 예측 정확도", loc="left", fontsize=12.0, fontweight="bold")

w = 0.36
short = ["포도상구균\n항균", "세레우스\n항균", "NO\n항염", "PGE2\n항염", "ABTS\n항산화"]
for k, (name, rnd, pred, lo, hi) in enumerate(res):
    b2.bar(k - w / 2 - 0.01, rnd, w, color=NEUTRAL, zorder=3)
    b2.bar(k + w / 2 + 0.01, pred, w, color=S1, zorder=3)
    b2.plot([k + w / 2 + 0.01] * 2, [lo, hi], color=INK2, lw=1.4, zorder=4)
    b2.text(k - w / 2 - 0.01, rnd + 0.2, f"{rnd:.1f}", ha="center", va="bottom", fontsize=8.4, color=INK2)
    b2.text(k + w / 2 + 0.01, hi + 0.25, f"{pred:.1f}", ha="center", va="bottom", fontsize=9.4, fontweight="bold",
color=INK)
b2.set_xticks(range(len(res)), short, fontsize=9.0)
b2.set_ylim(0, 17.5)
b2.tick_params(labelsize=8)
b2.set_ylabel("A등급 자원 수 (건)", fontsize=10.2)
b2.grid(axis="y", color=GRID, lw=0.8)
b2.set_axisbelow(True)
for s_ in ("top", "right"):
    b2.spines[s_].set_visible(False)
b2.legend(handles=[Patch(color=NEUTRAL, label="임의 50주 (기대값)"), Patch(color=S1, label="예측 상위 50주 (평균, 선은 최소~최대)")],
          loc="upper center", bbox_to_anchor=(0.5, 1.0), frameon=False, fontsize=7.8, ncol=2, columnspacing=1.2,
handlelength=1.4)
b2.set_title("(나) 연간 미생물 한도 50주 안의 A등급 수", loc="left", fontsize=12.0, fontweight="bold")
fig.suptitle("그림 3. 계통 기반 예측의 정확도와 분양 한도 안에서의 효과", x=0.01, ha="left", fontsize=14.4,
fontweight="bold")
fig.text(0.01, 0.01, "학습에 쓰지 않은 종을 맞히는 조건(종 단위 5겹 교차검증을 10회 반복). (나)는 미생물 추출물 대상 모의 비교.", fontsize=8.4, color=MUTED)
fig.tight_layout(rect=(0, 0.035, 1, 0.965))
fig.savefig(os.path.join(RESULT, "그림3_예측정확도_모의비교.png"))
plt.close(fig)
print("그림 저장 완료 →", RESULT)
```

수집/scrape_mbris.py · MBRIS 효능 뱅크 표 수집(수집 방법 기록) · 39줄

```
"""MBRIS 효능 뱅크 목록 수집 (서버 렌더링 테이블, pageUnit=100, 요청 간 1초 대기)."""
import re, sys, time, html, csv, urllib.request, urllib.parse

sys.stdout.reconfigure(encoding="utf-8")
OUT = sys.argv[1]
BANKS = sys.argv[2:] or ["extractBeautyBank", "extractAntiBac", "extractMtblsmDiss"]
UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/128"


def fetch(bank, page):
    data = urllib.parse.urlencode({"pageIndex": page, "pageUnit": 100}).encode()
    req = urllib.request.Request(f"https://www.mbris.kr/biobank/{bank}/search.do", data=data, headers={"User-Agent": UA})
    with urllib.request.urlopen(req, timeout=60) as r:
        return r.read().decode("utf-8", "replace")


def cells(tr):
    tds = re.findall(r"<td[^>]*>(.*?)</td>", tr, re.S)
    return [re.sub(r"\s+", " ", html.unescape(re.sub(r"<[^>]+>", " ", t))).strip() for t in tds]


for bank in BANKS:
    first = fetch(bank, 1)
    total = int(re.search(r"총 ([\d,]+)건", first).group(1).replace(",", ""))
    pages = (total + 99) // 100
    rows = []
    for p in range(1, pages + 1):
        s = first if p == 1 else fetch(bank, p)
        tbs = re.findall(r"<tbody[^>]*>(.*?)</tbody>", s, re.S)
        for tb in tbs:
            for tr in re.findall(r"<tr[^>]*>(.*?)</tr>", tb, re.S):
                c = cells(tr)
                if any(x.startswith("MABIK") for x in c):
                    rows.append(c)
        print(bank, p, "/", pages, len(rows), flush=True)
        time.sleep(1)
    with open(f"{OUT}/{bank}.csv", "w", newline="", encoding="utf-8-sig") as f:
        csv.writer(f).writerows(rows)
    print("DONE", bank, total, len(rows), flush=True)
```

수집/ncbi_lineage.py · NCBI Taxonomy 계통 수집(수집 방법 기록) · 76줄

```python
"""미생물·미세조류 속(genus)의 NCBI Taxonomy 계통(문·강·목·과)을 받아 ../데이터/ncbi_genus_lineage.json 으로 저장.

NCBI E-utilities 사용 지침에 따라 초당 3회 이하로 요청한다. 이미 저장된 결과 파일이 제출 자료에 들어 있으므로
분석 재현에는 이 스크립트를 다시 실행할 필요가 없다(수집 방법을 기록하기 위한 용도).

실행: python ncbi_lineage.py
"""
import json
import os
import re
import sys
import time
import urllib.parse
import urllib.request
import xml.etree.ElementTree as ET

import pandas as pd

sys.stdout.reconfigure(encoding="utf-8")
DATA = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "..", "데이터")
E = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/"

genera = set()
for fn in ("bank_microbe.csv", "bank_microalgae.csv"):
    d = pd.read_csv(os.path.join(DATA, fn), header=None, encoding="utf-8-sig")
    genera |= set(d[1].astype(str).str.extract(r"^([A-Z][a-z]+)")[0].dropna())
genera = sorted(genera)
print("속 개수", len(genera), flush=True)


def get(url):
    for _ in range(3):
        try:
            with urllib.request.urlopen(url, timeout=30) as r:
                return r.read().decode("utf-8")
        except Exception:
            time.sleep(2)
    return ""


ids = {}
for i, g in enumerate(genera):
    q = urllib.parse.quote(f"{g}[Scientific Name] AND genus[Rank]")
    s = get(f"{E}esearch.fcgi?db=taxonomy&term={q}&retmode=json")
    try:
        lst = json.loads(s)["esearchresult"]["idlist"]
        if lst:
            ids[g] = lst[0]
    except Exception:
        pass
    time.sleep(0.35)
    if i % 100 == 0:
        print("검색", i, len(ids), flush=True)

out = {}
items = list(ids.items())
for k in range(0, len(items), 100):
    chunk = items[k:k + 100]
    x = get(f"{E}efetch.fcgi?db=taxonomy&id={','.join(t for _, t in chunk)}&retmode=xml")
    try:
        root = ET.fromstring(x)
    except Exception:
        continue
    for tx in root.findall("Taxon"):
        name = tx.findtext("ScientificName")
        lin = {}
        for a in tx.findall("LineageEx/Taxon"):
            rk = a.findtext("Rank")
            if rk in ("phylum", "class", "order", "family"):
                lin[rk] = a.findtext("ScientificName")
        out[name] = [lin[r] for r in ("phylum", "class", "order", "family") if r in lin]
    time.sleep(0.4)
    print("조회", k, len(out), flush=True)

json.dump(out, open(os.path.join(DATA, "ncbi_genus_lineage.json"), "w", encoding="utf-8"), ensure_ascii=False)
print("완료", len(genera), len(ids), len(out))
```